

Named universal character escapes

Document #: P2071R1
Date: 2021-11-09
Project: Programming Language C++
Audience: SG16
EWG
Reply-to: Tom Honermann
<tom@honermann.net>
R. Martinho Fernandes
<rmf@mozilla.com>
Peter Bindels
<dascandy@gmail.com>
Corentin Jabot
<corentin.jabot@gmail.com>
Steve Downey
<sdowney@gmail.com, sdowney2@bloomberg.net>

Contents

- [1 Abstract](#)
- [2 Introduction](#)
- [3 Changes since R0](#)
- [4 Proposal](#)
- [5 History](#)
- [6 Motivation](#)
- [7 Design considerations](#)
 - [7.1 Syntax](#)
 - [7.2 Name sources](#)
 - [7.3 Name matching](#)
 - [7.4 Portable names](#)
 - [7.5 Existing practice](#)
 - [7.6 Backward compatibility](#)
 - [7.7 Implementor impact](#)
 - [7.8 Design alternatives](#)
- [8 Possible future extensions](#)
- [9 Implementation experience](#)
- [10 Acknowledgements](#)

- [11 Wording](#)
- [12 References](#)

§ 1 Abstract

A proposal to extend universal character names from hexadecimal sequences to include the official names and formal aliases of Unicode codepoints.

§ 2 Introduction

This proposal continues the effort R. Martinho Fernandes initiated that culminated in [\[P1097R2\]](#) “Named character escapes”. This proposal does not deviate from the general design intent in Fernandes’ work, but does deviate in a few details. See the [History](#) and [Proposal](#) sections for more information.

C++ programmers have been able to portably use characters outside of the basic source character set in character and string literals since the introduction of [universal-character-names](#) in C++11. For example:

```
// UTF-32 character literal with U+0100 {LATIN CAPITAL LETTER A WITH MACRON}
U'\u0100'
// UTF-8 string literal with U+0100 {LATIN CAPITAL LETTER A WITH MACRON} U+0300
{COMBINING GRAVE ACCENT}
u8"\u0100\u0300"
```

This proposal enables the above literals to be written using Unicode assigned names instead of Unicode code point values.

```
U'\N{LATIN CAPITAL LETTER A WITH MACRON}' // Equivalent to U'\u0100'
u8"\N{LATIN CAPITAL LETTER A WITH MACRON}\N{COMBINING GRAVE ACCENT}" // Equivalent to
u8"\u0100\u0300"
```

This paper discusses and links to work completed by Corentin Jabot, [\[CJ-IMPL\]](#), that investigates implementation impact, though an implementation has not shipped in GGC or Clang. This paper also includes discussion regarding alternative design possibilities.

§ 3 Changes since R0

- Recommend universal-character-names be extended, as from [\[P2290\]](#) is in final review.
- Updated the proposal to match the EWG design consensus reached in Prague. Removed the proposal options section.
- Moved some content previously in the introduction section into a new history section.
- Added results of SG16 and EWG polls taken in Prague.
- Updated the existing practice section to correctly describe the name matching behavior of other languages where the behavior was previously uncertain.
- Updated uses of U+NNNN to correctly follow Unicode notational conventions.

§ 4 Proposal

The wording included in this proposal is for the following design:

- Context:

- “Named character escapes” are an alternative form of universal character names, allowed anywhere a UCN is.
- Syntax:
 - `\N{xxx}` where `xxx` is the name of the character.
- Name sources:
 - ISO/IEC 10646 assigned names.
 - ISO/IEC 10646 formal aliases.
 - Unicode Character Database names for C0 and C1 control characters.
 - No allowance for additional implementation-defined names.
- Name matching:
 - case-sensitive and whitespace-sensitive exact matches.
- Feature test macro:
 - `__cpp_named_character_escapes`

§ 5 History

Prior presentations of P1097 to EWG-I and EWG received strong encouragement and useful design feedback:

- Review of [P1097R1] by [EWG-I in San Diego, 2018](#):

Do we want named escape sequences?

SF	F	N	A	SA
5	9	7	0	0

Do we want to support name aliases?

SF	F	N	A	SA
12	8	1	0	0

Do we want case-insensitive matching?

SF	F	N	A	SA
5	7	4	4	1

Do we want full UAX #44 LM2 name matching?

SF	F	N	A	SA
0	0	7	7	7

- Review of [P1097R2] by [EWG in Belfast, 2019](#):

EWG wants to encourage further work in this area

SF	F	N	A	SA
8	16	8	1	1

Motion passes

Accept P1097 as presented for C++23

SF	F	N	A	SA
2	9	13	5	1

No consensus. Author encouraged to do further work

Two areas of concern were raised during [discussion in EWG in Belfast, 2019](#):

- **Implementation impact** The Unicode name database (names and aliases), in text form, is ~1.5 MiB and a naive implementation could significantly impact the size of compiler distributions. This was of particular concern to organizations that distribute compilers as part of a distributed build process.
- **Design concerns** One EWG member strongly preferred a library based design that would have a smaller impact on the core language. For example, a string interpolation based design.

The implementation concerns prompted Corentin Jabot to explore implementation strategies as described in the [Implementation experience](#) section.

Despite the clear negative feedback from EWG-I with regard to use of [\[UAX44-LM2\]](#) to match character names, [\[P2071R0\]](#) proposed using [\[UAX44-LM2\]](#). This was motivated solely by Corentin Jabot's use of that algorithm in his implementation experiments.

Presentation of P2071R0(<https://wg21.link/p2071r0>) to SG16 and EWG in Prague received strong encouragement and consensus for design direction.

- [SG16 in Prague, 2020](#):

What is our preferred name matching algorithm?

In favor	Name match algorithm
6	Exact match.
6	Case insensitive
4	Full UAX44-LM2

No consensus for the UAX44-LM2 algorithm.

We should support case-insensitive matching as opposed to exact match?

SF	F	N	A	SA
2	3	2	1	2

Consensus? No

- SF: Matches implementations in other languages.
- SF: Mixed case is more legible than UPPERCASE.
- SA: This is an identifier in a case-sensitive language.
- SA: Increases maintenance in large code bases due to different style preferences; want one way to spell things.
- N: Want UAX44-LM2 because I'll constantly have to lookup correct names.

Preferred syntax: (vote for 1)

In favor	Syntax
8	Use “\N{XXX}”
0	Use “\u{XXX}” and “\U{XXX}”

Strong consensus for the originally proposed syntax.

- F: Want to reserve other potential extensions
- F: Matches other languages like Python.

Match name aliases?

SF	F	N	A	SA
8	2	0	0	0

Consensus? Yes

Include support for ISO/IEC 10646 named sequences?

SF	F	N	A	SA
0	0	1	6	1

Consensus? No - SA: Adds implementation complexity for little benefit. - A: Can be added later.

Forward to EWG with: no UAX44-LM2 matching, no support for named sequences, use of \N, and no recommendation regarding case-sensitivity.

SF	F	N	A	SA
7	3	0	0	0

Consensus? Yes

- [EWG in Prague, 2020](#):

We are interesting in supporting named universal character escapes

SF	F	N	A	SA
14	5	0	0	0

This should further support aliases

SF	F	N	A	SA
18	2	1	0	0

It should further be case insensitive

SF	F	N	A	SA
0	6	6	9	2

It should further support UAX44-LM2 with arbitrary spaces and dashes

SF	F	N	A	SA
1	4	5	8	5

Here again, clear negative feedback was provided with regard to use of the [UAX44-LM2] name matching algorithm. Additionally, the clearest guidance obtained so far was provided with regard to case-insensitivity. Corentin Jabot experimented and found that use of [UAX44-LM2] reduced data size by about 9K; roughly 5%, which is not insignificant.

Revision P2071R1 has been modified to match the EWG consensus to require exact name matches only.

§ 6 Motivation

The introduction of *universal-character-names* in C++11 benefitted programmers by allowing them to portably encode characters outside of the basic source character set without having to resort to use of octal or hexadecimal *escape-sequences* to explicitly encode code units. However, Unicode code points by themselves do not clearly communicate to readers of the code which character is to be encoded; hence the code comments included with the code examples in the introduction. Allowing programmers to directly use Unicode assigned character names avoids the need for side channel communications, like code comments, that might get out of sync over time.

Use of UTF-8 as the encoding for source files has increased over time, but impediments to adoption remain. For example, Microsoft Visual C++ still defaults to a locale dependent encoding and that encourages limiting source files to ASCII. If the C++ community were to migrate en masse to UTF-8, then one might question whether *universal-character-names* would become a legacy backward compatibility feature since programmers could reliably type the intended character in their source code directly. And if *universal-character-names* were to become an anachronism, then what use would be served by introducing a named character escape?

Unicode defines a number of characters that, even when they can be typed directly, can result in confusion. These include invisible characters such as U+200B {ZERO WIDTH SPACE}, combining characters such as U+0300 {COMBINING GRAVE ACCENT}, visually indistinct characters such as U+003B {SEMICOLON} and U+037E {GREEK QUESTION MARK}, and characters with RTL (right-to-left) directionality. Consider how the following string literals containing these characters are rendered. In cases like these, use of escape sequences improves clarity; thus motivation for use of Unicode escape sequences will remain.

" "	// U+200B {ZERO WIDTH SPACE}
" "	// U+200F {RIGHT-TO-LEFT MARK}
" "	// U+0300 {COMBINING GRAVE ACCENT}
" ; "	// U+003B {SEMICOLON}
" ; "	// U+037E {GREEK QUESTION MARK}
" ' "	// U+00B4 {ACUTE ACCENT}
" "	// U+0301 {COMBINING ACUTE ACCENT}
" ' "	// U+1FFD {GREEK OXIA}
"Ω"	// U+03A9 {GREEK CAPITAL LETTER
"Ω"	OMEGA}
"Α"	// U+2126 {OHM SIGN}
"Α"	// U+0041 {LATIN CAPITAL LETTER A}
"Α"	// U+0391 {GREEK CAPITAL LETTER
"Α"	ALPHA}
"Α"	// U+0410 {CYRILLIC CAPITAL LETTER A}
"Α"	// U+13AA {CHEROKEE LETTER GO}
"Α"	// U+A4EE {LISU LETTER A}

```
// U+102A0 {CARIAN LETTER A}  
// U+16F40 {MIAO LETTER ZZYA}
```

Named character escapes are supported in various forms in other programming languages. The following is the result of a brief survey of various languages. For languages that include such support, more details can be found in the [Design considerations](#) section.

Language	Named character escape support
C#	No
D	Yes; HTML 5 named character references
Go	No
Java	No
Javascript	No
Perl	Yes; Unicode names, aliases, and named sequences
PHP	No
Python	Yes; Unicode names and aliases
Raku	Yes; Unicode names, aliases, named sequences, and emoji sequences
Ruby	No
Rust	No
Swift	No
Visual Basic	No

§ 7 Design considerations

There are numerous choices for how support for named characters can be integrated into C++. Useful questions for making design choices include:

- Which names will be recognized? Can multiple names for the same character exist?
- How will names be matched? Must they be exact? Case insensitive?
- How will support for new names affect backward compatibility?
- How will the requirement for a name database impact implementations?
- What syntax to use?
- What is existing practice in other languages?

This section analyzes the various options considered for this proposal.

§ 7.1 Syntax

Named character escapes are proposed as a more readable alternative to [universal-character-names](#). As such, it is desirable that they be similar in syntax to [universal-character-names](#) and other existing escape sequences.

The syntax proposed by Fernandes in [\[P1097R2\]](#) “Named character escapes”) is modeled after the syntax adopted for Python and consists of a `\N` escape introducer followed by a name enclosed in curly brackets. For example:

```
'\N{LATIN CAPITAL LETTER A}'
```

```
"\N{LATIN CAPITAL LETTER A WITH MACRON}"
```

Other choices for the escape introducer are possible; the [Backward compatibility](#) section discusses some possible motivation for preferring `\u` and/or `\U`.

Options for recognized names and how to match them are discussed in subsequent sections.

As proposed, only one name is allowed per named character escape, but that is an artificial limitation. Raku allows a sequence of comma separated names to be specified in a single escape. This is a natural extension if names are permitted to identify sequences of characters instead of a single character. The following would all be equivalent. This proposal leaves this option to a future extension; see the [Possible future extensions](#) section.

```
"\N{LATIN CAPITAL LETTER A WITH MACRON, COMBINING GRAVE ACCENT}"  
"\N{LATIN CAPITAL LETTER A WITH MACRON}\N{COMBINING GRAVE ACCENT}"  
"\u0100\u0300"
```

Perl and Raku both allow Unicode code point numbers to be specified as character names. Following suit would enable a syntax that avoids the strict 4 or 8 digit requirements of [universal-character-names](#) and could allow the natural `U+NNNN` style frequently used to identify Unicode characters. The following could all be equivalent. This proposal also leaves this option for a future extension as discussed in the [Possible future extensions](#) section.

```
"\N{U+0100}"  
"\N{U+100}"  
"\N{U+000100}"  
"\N{0x0100}"  
"\N{256}"  
"\u0100"
```

With the addition of forms for universal character names, such as for “LATIN CAPITAL LETTER A WITH MACRON”, it is straightforward to extend the syntax to support:

```
std::string \N{LATIN CAPITAL LETTER A WITH MACRON} = "\N{LATIN CAPITAL LETTER A  
WITH MACRON}";
```

§ 7.2 Name sources

A named character escape feature is not particularly useful unless accompanied by at least one source of character names. The following list contains sources of character names that are consulted by at least one implementation of named character escapes in another programming language.

- [Unicode assigned names \(synchronized with ISO/IEC 10646\)](#)
- [Unicode aliases \(synchronized with ISO/IEC 10646\)](#)
- [Unicode named sequences \(synchronized with ISO/IEC 10646\)](#)
- [Emoji ZWJ sequences](#)
- [Emoji sequences](#)
- [HTML named character references](#)

The first three are defined by the Unicode Consortium, part of the Unicode standard, and synchronized with ISO/IEC 10646. The names specified in each are designed in concert, share a common namespace, are immutable once published, and Unicode guarantees no conflicts between them. See the [Unicode character encoding stability policy](#) for more details. There is some loss of precision, however, in the code

point charts published in ISO 10646 and the underlying data files in the Unicode Character Database, in particular with respect to aliases for control characters. The base UCD sources are consulted for named character escapes in Perl, Python, and Raku.

The next two sources specify emoji character sequences. Though produced by the Unicode Consortium, they are not part of the Unicode standard, and are not covered by the [Unicode character encoding stability policy](#). These two sources don't technically provide names; they provide optional descriptions. The provided descriptions use characters, particularly `:` and `,`, that are disallowed in the names provided by the first three sources. These sources are consulted for named character escapes in Raku.

The last source is the specification of names recognized for use as named character references in HTML documents. This source is used for the implementation of named character escapes in the D programming language.

The stability guarantees offered by the Unicode standard are a strong motivator for their use and, as such, this proposal adopts them as the name sources to use.

The list of Unicode assigned names associates at most one associated name with each character. There may be more than one alias assigned. There are some characters that are not assigned a name in this list, for example, U+0080 is simply listed as a `<control>` character with no name. In some of these cases, the Unicode aliases list provides one or more names. For example, U+0080 has assigned aliases of `PADDING CHARACTER` (a figment alias) and `PAD` (an abbreviation alias). U+0009 has aliases of `CHARACTER TABULATION` and `HORIZONTAL TABULATION`. The software that is used to prepare the code point charts in the standard selects one as the formal alias, but it is not clear that this is entirely intentional or stable. The wording proposed takes the `<control>` aliases from the Unicode Character Database since at least Unicode 6, and places those aliases normatively in the C++ standard.

We are working with the Unicode Consortium and the ISO project for 10646 to improve the situation.

Unicode aliases provide another critical service. As mentioned above, once assigned, names are immutable. Corrections are only offered by providing an alias. Aliases, according to the NamedAliases tables in the Unicode Character Database, come in five varieties:

- **correction** Aliases for cases where an incorrect assigned name was published. For example, U+FE18 has an assigned name of `PRESENTATION FORM FOR VERTICAL RIGHT WHITE LENTICULAR BRAKCT` and a correction alias of `PRESENTATION FORM FOR VERTICAL RIGHT WHITE LENTICULAR BRACKET` (note the typo correction).
- **control** Aliases for various control characters. For example, `NULL` for U+0000.
- **alternate** Aliases for widely used alternate names. For example, `BYTE ORDER MARK` for U+FEFF.
- **figment** Aliases for names that were documented, but never accepted in a standard. For example, `HIGH OCTET PRESET` for U+0081.
- **abbreviation** Aliases for common abbreviations. For example, `NBSP` for U+00A0.

The intent is to use the aliases classified as `correction`, `control`, and `alternate` as recognized names.

It is conceivable that implementors could desire, or be requested to, support additional implementation-defined names; perhaps including from the additional sources listed above. Since new characters and names will continue to be added to the Unicode standard, caution is warranted to avoid the possibility of introducing conflicting names over time. The description of the [\[UAX44-LM2\]](#) name matching algorithm describes a historical case of how such a conflict once occurred. Any support for additional names should ensure that they occupy a non-overlapping namespace with the Unicode assigned names. Out of caution, this proposal disallows additional implementation-defined names.

§ 7.3 Name matching

Names can be finicky things. Having to remember whether a name is, for example, `ZERO WIDTH SPACE` or `ZERO-WIDTH SPACE` is likely to frustrate programmers. Some programmers might prefer `zero width space`.

Unicode provides a straight forward algorithm for matching names with various allowances including case-insensitivity, omission of some hyphens (-), and substitution of underscore (_) for space characters. [UAX44-LM2] is included in the Unicode standard via [UAX44].

The [UAX44-LM2] matching rule would accept any of the following names as a match for U+200B {ZERO WIDTH SPACE}

```
ZERO WIDTH SPACE
ZERO-WIDTH SPACE
zero-width space
ZERO width S P_A_C E
```

[UAX44] 5.9.2 (https://www.unicode.org/reports/tr44/tr44-24.html#Matching_Names) explicitly recommends that names *should* be matched with the loose algorithm:

While each Unicode character name for an assigned character is guaranteed to be unique, names are assigned in such a way that the presence or absence of spaces cannot be used to distinguish them. Furthermore, implementations sometimes create identifiers from Unicode character names by inserting underscores for spaces. For best results in comparing Unicode character names, use loose matching rule UAX44-LM2.

However, this recommendation is widely ignored by programming languages. Perl uses LM2. Most use either exact match, or case folding. See [Existing Practice](#).

This proposal uses strict matching both for case and whitespace. This preserves the option to relax matching later.

§ 7.4 Portable names

Portably using named character escapes will require implementations to agree on a minimum version of the name sources.

Thanks to the adoption of [P1025R1]“Update The Reference To The Unicode Standard”) in Rapperswil, 2019, the C++ standard has a normative floating reference to [ISO/IEC 10646](#) “Information technology — Universal Coded Character Set (UCS)”, the ISO/IEC standard that specifies a subset of what is specified in the Unicode standard and the assigned character set names and numbers are kept in synchronization with it. ISO/IEC 10646:2020 includes the Unicode assigned names (in section 34, Code Charts and lists of character names), name aliases (also in section 34), and named character sequences (in section 28, Named UCS Sequence Identifiers), which cross references to a machine readable format at <https://standards.iso.org/iso-iec/10646/ed-6/en/NUSI.txt>

The floating reference to ISO/IEC 10646 indicates a dependence on the version that is current at the time of standardization. Thus, conformance with the C++ standard will require conformance with the latest available publication of ISO/IEC 10646.

Implementors must be allowed, and encouraged, to conform to more recent versions of ISO/IEC 10646 as they are published, or to use names from the Unicode standard as those standards are published.

§ 7.5 Existing practice

Support for named escape sequences exists in several programming languages. The following details of existing practice were obtained from these documentation sources.

Language	Documentation link
D	https://dlang.org/spec/lex.html#StringLiteral
Perl	https://perldoc.perl.org/charnames.html
Python	https://docs.python.org/3.8/reference/lexical_analysis.html#literals
Raku	https://docs.raku.org/language/unicode#Entering_unicode_codepoints_and_codepoint_sequences

Capabilities vary across languages:

Language	Name sources	Comma separated names	Name matching	Matches code point numbers
D	HTML 5	No	Case-sensitive and whitespace-sensitive.	No
Perl	Unicode names Unicode name aliases Unicode named sequences registered custom aliases	No	By default, case-sensitive and whitespace-sensitive exact match. Optionally, script qualified short names with use <code>charnames 'short';</code> . Optionally, UAX44-LM2 with use <code>charnames 'loose';</code> (case insensitive, ignore underscore, most spaces, and most non-medial hyphens)	Yes
Python	Unicode names Unicode name aliases	No	Case-insensitive, but whitespace-sensitive	No

Language	Name sources	Comma separated names	Name matching	Matches code point numbers
Raku	Unicode names Unicode name aliases Unicode named sequences emoji ZWJ sequences emoji sequences	Yes	Case-insensitive, but whitespace-sensitive	Yes

Examples:

Language	Code
D	"\&Aacr;"
Perl	"\N{LATIN CAPITAL LETTER A WITH MACRON}" "\N{U+0100}"
Python	"\N{LATIN CAPITAL LETTER A WITH MACRON}"
Raku	"\c[LATIN CAPITAL LETTER A WITH MACRON]" "\c[256]" "\c[LATIN CAPITAL LETTER A WITH MACRON, COMBINING GRAVE ACCENT]" "\c[LATIN CAPITAL LETTER A WITH MACRON AND GRAVE]"

§ 7.6 Backward compatibility

Escape sequences beyond those required in the standard are conditionally-supported ([\[lex.ccon\]p7](#)). For implementations that currently define a meaning for `\N` in character or string literals, the use of `\N` in this proposal is technically a breaking change.

Gcc, Clang, and Microsoft Visual C++ all accept `\N` as an escape sequence with the semantic effect of substituting `N` such that `"\N{xxx}"` is equivalent to `"N{xxx}"`. However, they each emit a warning regarding an unrecognized escape sequence, so reliance on this behavior is not likely to be common. Still, there are likely to be some uses in the wild (probably some percentage of that were intended to be `\n`).

Another option would be to reuse the `\u` and/or `\U` introducer used for [universal-character-names](#). Gcc and Clang both reject code like `"\u{xxx}"` and `"\U{xxx}"` as containing ill-formed [universal-character-names](#). However, Microsoft Visual C++ accepts such uses without a warning and treats them as equivalent to `"u{xxx}"` and `"U{xxx}"` respectively. There is an in progress proposal to extend universal character names to include `\u{xxx}` forms. This proposal avoids conflicts with that syntax.

The implementation divergence that occurs for the `\u` and `\U` cases above suggests that repurposing them may reduce the potential for backward compatibility impact. Use of `\u` and/or `\U` would potentially require more wording changes to distinguish named character escapes from [universal-character-names](#), but would be unlikely to pose a significant additional impact to implementors.

For now, this proposal adheres to Fernandes' original design and retains use of `\N` as the introducer for named character escapes.

§ 7.7 Implementor impact

The sources of character names listed in the [Name sources](#) section do not constitute big data by today's standards, but that does not mean that the volume of data and potential for impact to compiler distributions and compiler performance is insignificant. As mentioned earlier, some organizations have valid technical reasons to be sensitive to the size of the compiler distributions they use; in a distributed build environment that distributes compilers, the size of the distribution impacts latency and can therefore negatively impact build times.

The combined size of the Unicode 12.0 text files containing the Unicode assigned names, aliases, and named character sequences is approximately 1.5 MiB. A naive implementation might contribute 2+ MiB of code/data to a compiler. Some EWG members indicated that amount of increase is a cause for concern.

Fortunately, naive implementations are not the only option. Corentin Jabot has done some excellent work to demonstrate that an implementation should be possible that increases the code/data size of a compiler by less than 300 KiB. See the [Implementation experience](#) section for details. Corentin's approach is promising, but the additional complexity carries additional implementation cost and maintenance.

Staying up to date with new Unicode releases will also, of course, pose an additional cost on implementors.

§ 7.8 Design alternatives

As indicated previously, at least one EWG member in Belfast was strongly interested in a more general core language feature, presumably a string interpolation facility, that would allow named character escapes to be implemented as a library feature. Such a feature could take many forms, but might look something like the following where `\{` is an escape sequence followed by a call to a `constexpr` function named `nce` with arguments passed in some form.

```
"\{nce(LATIN CAPITAL LETTER A WITH GRAVE)}"
```

Such a feature could certainly be implemented, but would seem to necessarily be more verbose and would necessitate inclusion of appropriate headers; headers that would be quite large in the case of a named character database or that would make use of a compiler intrinsic; which would put the complexity back in the compiler (though in implementation-defined territory rather than in standard core language). The verbosity concern could potentially be reduced by introducing core language sugar for lowering the proposed syntax to the example string interpolation syntax above.

§ 8 Possible future extensions

The following options are *not* currently proposed but could be considered for future extension.

1. Allow comma separated names. For example:

- `"\N{LATIN CAPITAL LETTER A WITH MACRON, COMBINING GRAVE ACCENT}" //`
Equivalent to `"\u0100\u0300"`
- 2. Allow code point numbers as names. For example:
 - `"\N{U+00C0}" //` Equivalent to `"\u00C0"`
 - `"\N{0x00C0}" //` Equivalent to `"\u00C0"`
 - `"\N{192}" //` Equivalent to `"\u00C0"`
- 3. Allow names to match ISO/IEC 10646 named sequences such that the following would be equivalent:
 - `"\N{LATIN CAPITAL LETTER A WITH MACRON AND GRAVE}"`
 - `"\N{LATIN CAPITAL LETTER A WITH MACRON}\N{COMBINING GRAVE ACCENT}"`
 - `"\u0100\u0300"`
- 4. Allow names to match Unicode emoji named sequences. For example:
 - `"\N{keycap: #}" //` Equivalent to `"\u0023\uFE0F\u20E3"`
 - `"\N{Czech Republic}" //` Equivalent to `"\U0001F1E8\u0001F1FF"`
 - `"\N{waving hand: medium skin tone}" //` Equivalent to `"\U0001F1E8\u0001F1FF"`
- 5. Allow names to match Unicode emoji ZWJ named sequences. For example:
 - `"\N{man shrugging: medium skin tone}" //` Equivalent to
`"\U0001F937\u0001F3FD\u200D\u2642\uFE0F"`
 - `"\N{rainbow flag}" //` Equivalent to
`"\U0001F3F3\uFE0F\u200D\u0001F308"`
- 6. Allow names to match HTML 5 named character references by surrounding them with `&` and `;`. For example:
 - `"\N{À}" //` Equivalent to `"\u00C0"`

§ 9 Implementation experience

This proposal been implemented in [Clang by Corentin Jabot](#), and in [Circle by Sean Baxter](#), based on the library work available at [blog post](#) on the experiment reported that he was able to implement a function (`cp_from_name`) that accepts a Unicode 12.0 name or name alias and returns a code point value in under 300 KiB. His implementation is available in the `cp_to_name` branch of his `ext-unicode-db` GitHub repository at https://github.com/cor3ntin/ext-unicode-db/tree/name_to_cp Baxter reports a footprint of 272K for adding the feature.

§ 10 Acknowledgements

Thank you to R. Martinho Fernandes for taking the initiative to research and first propose support for named character escapes and for contributing his considerable expertise in general to SG16.

Thank you to Corentin Jabot for the excellent work he did experimenting with and analyzing implementation impact. Without his work, the data necessary to respond to the implementation concerns raised in Belfast would not have been available at this time, thereby delaying further progress on this proposal.

Thank you to Peter Bindels and Corentin Jabot for providing feedback on an initial draft that I delivered to them less than two hours before the Prague pre-meeting mailing deadline!

§ 11 Wording

These changes are relative to [N4901] “Working Draft, Standard for Programming Language C++”

Modify [lex.charset]

(lex.charset.3) The universal-character-name construct provides a way to name other characters.

n-char: one of

A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
0 1 2 3 4 5 6 7 8 9
U+002D HYPHEN-MINUS
U+0020 SPACE

n-char-sequence:

n-char
n-char n-char-sequence

named-universal-character:

`\N { n-char-sequence }`

hex-quad:

hexadecimal-digit hexadecimal-digit hexadecimal-digit hexadecimal-digit

simple-hexadecimal-digit-sequence:

hexadecimal-digit
simple-hexadecimal-digit-sequence hexadecimal-digit

universal-character-name:

`\u hex-quad`
`\U hex-quad hex-quad`
named-universal-character

A *universal-character-name* of the form `\u hex-quad` or `\U hex-quad hex-quad` designates the character in the translation character set whose UCS scalar value is the hexadecimal number represented by the sequence of *hexadecimal-digits* in the *universal-character-name*. The program is ill-formed if that number is not a UCS scalar value. A *named-universal-character* designates the character in the translation character set whose associated character name or character name alias is the given *n-char-sequence*. The program is ill-formed if there is no such character.

In addition to the associated character name and the character name alias defined in 10646:2020 section 34, the following aliases are provided for control characters which otherwise have no formal name or alias.

Note: These names are derived from the Unicode Character Database’s CharacterAliases.txt. For historical reasons the C0 and C1 control characters are formally unnamed.

Codepoint	Alias
U+0000	NULL
U+0001	START OF HEADING
U+0002	START OF TEXT
U+0003	END OF TEXT
U+0004	END OF TRANSMISSION
U+0005	ENQUIRY
U+0006	ACKNOWLEDGE
U+0007	ALERT
U+0008	BACKSPACE
U+0009	CHARACTER TABULATION
U+0009	HORIZONTAL TABULATION
U+000A	LINE FEED
U+000A	NEW LINE
U+000A	END OF LINE
U+000B	LINE TABULATION
U+000B	VERTICAL TABULATION
U+000C	FORM FEED
U+000D	CARRIAGE RETURN
U+000E	SHIFT OUT
U+000E	LOCKING-SHIFT ONE
U+000F	SHIFT IN
U+000F	LOCKING-SHIFT ZERO
U+0010	DATA LINK ESCAPE
U+0011	DEVICE CONTROL ONE
U+0012	DEVICE CONTROL TWO
U+0013	DEVICE CONTROL THREE
U+0014	DEVICE CONTROL FOUR
U+0015	NEGATIVE ACKNOWLEDGE
U+0016	SYNCHRONOUS IDLE
U+0017	END OF TRANSMISSION BLOCK
U+0018	CANCEL
U+0019	END OF MEDIUM
U+001A	SUBSTITUTE
U+001B	ESCAPE
U+001C	INFORMATION SEPARATOR FOUR
U+001C	FILE SEPARATOR
U+001D	INFORMATION SEPARATOR THREE
U+001D	GROUP SEPARATOR
U+001E	INFORMATION SEPARATOR TWO
U+001E	RECORD SEPARATOR
U+001F	INFORMATION SEPARATOR ONE
U+001F	UNIT SEPARATOR
U+007F	DELETE

Codepoint	Alias
U+0082	BREAK PERMITTED HERE
U+0083	NO BREAK HERE
U+0084	INDEX
U+0085	NEXT LINE
U+0086	START OF SELECTED AREA
U+0087	END OF SELECTED AREA
U+0088	CHARACTER TABULATION SET
U+0088	HORIZONTAL TABULATION SET
U+0089	CHARACTER TABULATION WITH JUSTIFICATION
U+0089	HORIZONTAL TABULATION WITH JUSTIFICATION
U+008A	LINE TABULATION SET
U+008A	VERTICAL TABULATION SET
U+008B	PARTIAL LINE FORWARD
U+008B	PARTIAL LINE DOWN
U+008C	PARTIAL LINE BACKWARD
U+008C	PARTIAL LINE UP
U+008D	REVERSE LINE FEED
U+008D	REVERSE INDEX
U+008E	SINGLE SHIFT TWO
U+008E	SINGLE-SHIFT-2
U+008F	SINGLE SHIFT THREE
U+008F	SINGLE-SHIFT-3
U+0090	DEVICE CONTROL STRING
U+0091	PRIVATE USE ONE
U+0091	PRIVATE USE-1
U+0092	PRIVATE USE TWO
U+0092	PRIVATE USE-2
U+0093	SET TRANSMIT STATE
U+0094	CANCEL CHARACTER
U+0095	MESSAGE WAITING
U+0096	START OF GUARDED AREA
U+0096	START OF PROTECTED AREA
U+0097	END OF GUARDED AREA
U+0097	END OF PROTECTED AREA
U+0098	START OF STRING
U+009A	SINGLE CHARACTER INTRODUCER
U+009B	CONTROL SEQUENCE INTRODUCER
U+009C	STRING TERMINATOR
U+009D	OPERATING SYSTEM COMMAND
U+009E	PRIVACY MESSAGE
U+009F	APPLICATION PROGRAM COMMAND

Change in table 17 of [15.11 \[cpp.predefined\]](#) paragraph 1.8:

Drafting note: the final value for the `__cpp_named_character_escapes` feature test macro will be selected by the project editor to reflect the date of approval.

Table 17 — Feature-test macros [tab:cpp.predefined.ft]	
Macro name	Value
[...]	[...]
<code>__cpp_modules</code>	201907L
<code>__cpp_named_character_escapes</code>	<u>XXXXXXL</u> ** placeholder **
<code>__cpp_namespace_attributes</code>	201411L
[...]	[...]

§ 12 References

[CJ-IMPL] Corentin Jabot. ext-unicode-db.

https://github.com/cor3ntin/ext-unicode-db/tree/name_to_cp

[N4901] Thomas Köppe. 2021-10-22. Working Draft, Standard for Programming Language C++.

<https://wg21.link/n4901>

[P1025R1] Steve Downey, JeanHeyd Meneide, Martinho Fernandes. 2018-06-07. Update The Reference To The Unicode Standard.

<https://wg21.link/p1025r1>

[P1097R1] R. Martinho Fernandes. 2018-06-22. Named character escapes.

<https://wg21.link/p1097r1>

[P1097R2] R. Martinho Fernandes. 2019-01-21. Named character escapes.

<https://wg21.link/p1097r2>

[P2071R0] Tom Honermann, Peter Bindels. 2020-01-13. Named universal character escapes.

<https://wg21.link/p2071r0>

[P2290] Corentin Jabot. Delimited escape sequences.

<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/p2290r2.pdf>

[UAX44] Ken Whistler and Laurențiu Iancu. Unicode Character Database.

<http://www.unicode.org/reports/tr44>

[UAX44-LM2] Ken Whistler and Laurențiu Iancu. Unicode Character Database LM2.

<https://www.unicode.org/reports/tr44/tr44-24.html#UAX44-LM2>